

A Governed Agentic AI Framework for Emergency Care

Matthew Lang
University of Virginia MSDS
School of Data Science
Charleston, SC, USA
vyv7zh@virginia.edu

Hannah Richardson
University of Virginia MSDS
School of Data Science
Houston, TX, USA
zhx9yf@virginia.edu

Heejeong (Angie) Yoon
University of Virginia MSDS
School of Data Science
Tampa, FL, USA
rpk3ve@virginia.edu

Chelsey Blowe
University of Virginia MSDS
School of Data Science
Chesapeake, VA, USA
qck2qg@virginia.edu

Abstract— In partnership with UVA Health, this capstone project focuses on developing the foundational infrastructure for a multi-year clinical AI agent initiative titled ForgeNFHIR. As the founding team, our primary objective was to establish a secure and scalable framework that supports real-time clinical decision making within the emergency department. The current phase of development involves the automated retrieval and synthesis of patient data to provide clinicians with an efficient overview of essential medical information. To enable safe and reproducible development, we leveraged synthetic Fast Healthcare Interoperability Resources (FHIR) data, initially focusing on allergy information as a high-impact use case for validating data retrieval, aggregation, and clinical relevance. We have also prioritized healthcare transparency and accountability by implementing a comprehensive auditing system for long term record retention. Additionally, we conducted research into different clinical output formats to determine how information can be best presented to various hospital stakeholders. This work provides the technical blueprint and data integrity standards required for the future transition to live clinical environments.

I. INTRODUCTION

The purpose of this project is to develop a clinical decision support tool that reduces the cognitive load on healthcare providers in the Emergency Department (ED). By automating the retrieval and synthesis of patient information into a concise clinical snapshot, the system enables clinicians to rapidly identify critical data such as allergies, active medications, recent encounters, and key clinical findings. In high-pressure ED environments, timely and accurate access to patient information is essential for ensuring patient safety and improving operational efficiency.

The proposed clinical agent serves as a first-line tool that provides a high-level overview of a patient’s history without requiring clinicians to manually navigate fragmented electronic health records. The system generates a structured ER Clinical Snapshot in less than 5 seconds, summarizing patient identity and relevant clinical context. Beyond direct clinical use, the platform supports administrative and auditing functions through the generation of structured data reports and execution traces.

The primary stakeholders for this project include UVA Health clinicians, hospital administrators, and future student teams who will extend the system’s capabilities. The current phase of development assumes that synthetic **Fast Healthcare Interoperability Resources (FHIR)** data, developed by HL7, provides a sufficient environment for testing the system’s architectural logic and safety constraints. It is also assumed that the secure scaffolding architecture—comprising FHIR server integration, a Model Context Protocol (MCP) layer, and a comprehensive local audit logging system—can be scaled to support more complex data types and additional clinical specialties.

The scope of this project includes the development of the secure infrastructure, implementation of the ER Clinical Snapshot workflow, and standardization of the development environment through a centralized dependency list and a formalized User Setup and Execution Guide. The audit logging system serves as a critical compliance mechanism by capturing trace data locally, enabling reproducibility and governance. Items currently out of scope include the integration of live patient data and the full implementation of multimodal capabilities, which are designated as long-term goals for future work.

A. Background: Analysis Tools

LangGraph is a system designed to build and orchestrate agent-based systems [3]. It has a framework where each node represents a well-defined function in the agent workflow (e.g., requirement parsing, validation, data retrieval, aggregation, and snapshot generation). This graph-based design enables explicit control over execution flow, supports checkpointing and interrupts, and ensures that intermediate states are observable and reproducible. The tool was chosen for its ability to support “infrastructure for any long running, stateful workflow or agent” (source). The system also supports easy modification, scalability, and deployment of agents.

Another major focus of analysis is the Model Context Protocol (MCP) as a standardized interface for data access. MCP replaces ad hoc query construction with structured tool invocation, enabling consistent retrieval of FHIR resources and

improving reproducibility across runs. This design also simplifies debugging and auditing by clearly separating reasoning steps from data access operations. The interaction between MCP tools and the LangGraph orchestration layer ensure that all external data dependencies remain transparent and controllable.

II. FHIR DATA ARCHITECTURE AND INTEGRATION

Fast Healthcare Interoperability Resources (FHIR), developed by HL7, provides a standardized framework for the representation and exchange of healthcare information [1]. In the Capstone_ER project, FHIR data forms the foundational layer of a multi-agent clinical intelligence system orchestrated using LangGraph and evaluated through LangSmith-style observability. Synthetic US Core-compliant datasets are loaded into a HAPI FHIR server [2] and accessed through both server-based queries and a local indexing adapter. These data are subsequently processed by specialized agents to generate an Emergency Department (ER) Clinical Snapshot, enabling timely and context-rich clinical insights while ensuring reproducibility, security, and compliance.

A. FHIR Standard and Interoperability

FHIR is a healthcare data standard designed to facilitate seamless information exchange across disparate clinical systems. By combining the best features of earlier HL7 standards with modern web technologies, FHIR supports RESTful APIs, standardized terminologies, and extensible resource definitions. Each FHIR resource represents a discrete clinical concept, enabling modular and scalable data integration.

Healthcare organizations use FHIR to support electronic health record (EHR) interoperability, clinical decision support, patient engagement applications, and population health analytics. The **US Core Implementation Guide** further constrains FHIR resources to ensure consistency and interoperability within the United States healthcare ecosystem. Key features of the FHIR standard include:

- **Resource-Based Architecture:** Encapsulates clinical entities such as patients and encounters.
- **RESTful API Support:** Enables real-time data retrieval using standard HTTP methods.
- **Terminology Standards:** Integrates coding systems such as SNOMED CT, LOINC, and ICD-10.
- **Extensibility:** Allows customization while maintaining interoperability.
- **Human-Readable and Machine-Processable Formats:** Supports JSON and XML representations.

These characteristics make FHIR an ideal foundation for AI-driven clinical intelligence systems such as the ER Clinical Snapshot.

B. FHIR Resource Types

FHIR data is organized into discrete resources, each representing a specific clinical or administrative concept. In the Capstone_ER project, several key resources were utilized to construct a comprehensive view of a patient’s emergency department history:

TABLE I. RESOURCE TYPE TABLE

Resource	Description	Role in ER Snapshot
Patient	Demographic and identifying information.	Provides patient context such as age and gender.
Encounter	Details of healthcare interactions.	Identifies emergency department visits and timelines.
Observation	Clinical measurements and laboratory results.	Supplies vital signs and diagnostic data.
MedicationRequest	Prescribed or administered medications.	Highlights active treatments and potential interactions.
AllergyIntolerance	Documented allergies and adverse reactions.	Supports patient safety and risk mitigation.
DiagnosticReport	Diagnoses and clinical problems.	Provides comorbidity and clinical history.
EpisodeOfCare	Longitudinal care management information.	Offers broader context for ongoing treatment.

C. Synthetic FHIR data

The Capstone_ER project employs synthetic FHIR data to enable system development and evaluation while safeguarding patient privacy. Synthetic datasets were generated using US Core-compliant tooling, producing realistic yet fully de-identified patient records that mimic the structure and statistical properties of real-world clinical data.

D. Source and Generation

Synthetic data were produced using a custom generation pipeline located in the FHIR Synthetic Data module of the repository. These datasets were then loaded into a HAPI FHIR server using the `clean_load_fhir_dataset.py` script, enabling standardized API-based access for the agentic workflow.

E. Rationale for Using Synthetic Data

The decision to utilize synthetic data as the foundational corpus for the ForgeNFHIR agent was driven by several

strategic considerations aimed at balancing technical innovation with institutional safety.

- **Privacy Preservation and HIPAA Compliance:** The primary driver for utilizing synthetic data is the absolute elimination of risk regarding Protected Health Information (PHI). By operating within a synthetic environment, the architecture can be developed and iterated upon without the legal and technical overhead of managing de-identification processes or risking accidental exposure. This "privacy-by-design" approach ensures a secure framework that adheres to strict HIPAA regulations from the earliest stages of the development lifecycle.
- **Ethical Research Practices:** Utilizing synthetic datasets enables rigorous experimentation and system stress-testing without the ethical complexities and administrative burdens of requiring individual patient consent. This allows for a more agile research environment, where the focus remains on architectural efficacy and clinical safety protocols before the system is ever introduced to a real-world patient care setting.
- **Scientific Reproducibility:** High-quality synthetic datasets are publicly shareable, allowing other researchers and future teams to replicate the study and verify the agent's performance. The use of standardized synthetic FHIR records ensures that the methodology is transparent and the results are independently verifiable by the broader academic community.
- **Development Flexibility:** Access to live clinical data is often gated by lengthy institutional review board (IRB) approval processes. Synthetic data provides an immediate, high-fidelity alternative that facilitates rapid prototyping. This flexibility allows the team to simulate a wide variety of clinical scenarios to refine the agent's reasoning engine and intent capture nodes in real-time.
- **Controlled Evaluation:** Synthetic data provides a consistent environment for benchmarking system performance. Unlike real-world data, which may contain irregular noise or non-standard coding, synthetic records allow for a precise evaluation of the agent's ability to parse specific FHIR resources. This controlled setting is essential for establishing baseline.

F. Limitations

While synthetic data closely resemble real clinical datasets, they may not fully capture the complexity and variability of real-world healthcare environments. As a result, system performance observed in this study may differ when deployed with live clinical data, highlighting the importance of future validation in operational settings.

III. SYSTEM ARCHITECTURE AND ORCHESTRATION

The core functionality of the clinical AI agent is driven by a sophisticated orchestration layer that bridges the gap between natural language processing and structured healthcare data. This architecture utilizes LangGraph for stateful agent orchestration and the Model Context Protocol for secure data retrieval.

To better understand how natural language inputs are parsed and routed through these systems to access clinical resources, a visual representation of the agentic workflow can be highly illustrative.

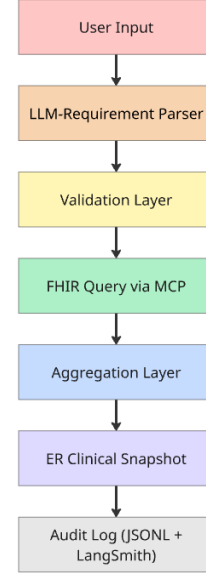


Fig. 1. System Architecture Flow Diagram

To demonstrate the system's functionality, consider a running example of a synthetic patient presenting to the emergency department. Before the agent can retrieve information, a local indexing adapter processes the raw synthetic FHIR data. When the US-Core dataset for this patient is ingested, the adapter specifically indexes critical resources such as AllergyIntolerance and MedicationRequest. This indexing transforms the complex JSON structure into a searchable database format, allowing the system to instantly locate the patient's severe penicillin allergy without scanning unrelated demographic data.

A. Orchestration and the Model Context Protocol

The system relies on multi-node architecture where specific tasks are delegated to specialized sub agents. The Model Context Protocol (MCP) acts as the vital bridge between the orchestration layer and the local FHIR server. This protocol ensures that when the agent interprets a clinician's request, it can securely and accurately query the underlying synthetic patient data without compromising the system's secure scaffolding.

Continuing with the patient example, when a clinician queries the system for the patient's allergies, the LangGraph orchestration layer does not access the database directly. Instead,

it utilizes the MCP to negotiate capability and access. The MCP acts as a secure bridge, validating that the agent has the correct permissions to request allergy data. Once verified, the MCP executes the search against the local indexing adapter and securely returns the penicillin allergy data back to the orchestration layer for synthesis.

B. Clinical Snapshot Generation

The primary output of the orchestration layer is the ER Clinical Snapshot. The snapshot generator node synthesizes the retrieved clinical resources into a concise format optimized for high velocity emergency department environments. The system processes complex inputs, such as requests for a specific patient's recent encounters, active medications, and allergy information, and transforms them into a structured output. The project evaluated multiple output formats, settling on list-based summaries for rapid clinical review and tabular data in JSON format as an alternative output for administrative stakeholders.

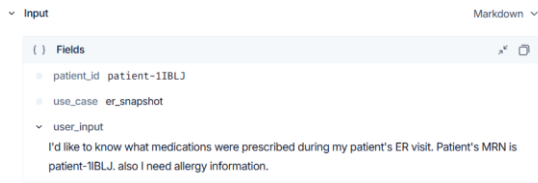


Fig. 2. Example of a ER Clinical Snapshot input.

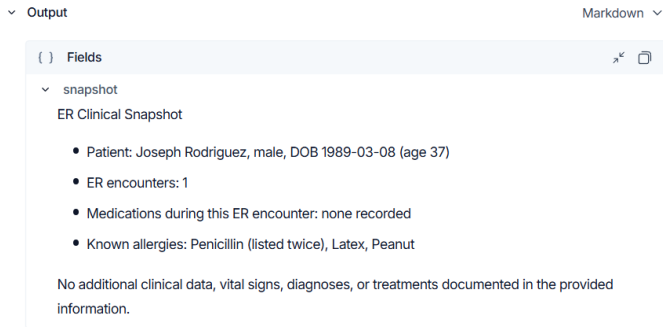


Fig. 3. Example of a ER Clinical Snapshot output.

IV. PIPELINE STANDARDIZATION AND AUDIT INFRASTRUCTURE

To ensure the clinical AI agent can be reliably deployed and governed, significant emphasis was placed on standardizing the development environment and implementing rigorous auditing mechanisms.

A. Environment Standardization and Setup

The team conducted a comprehensive review of the entire coding pipeline to identify and document specific errors and edge cases encountered during local deployment. This assessment resulted in the creation of a formalized User's Setup Guide for current and future users. Furthermore, to streamline the environment configuration process for new devices, the

team created a centralized requirements.txt file for the repository. This file houses all necessary Python libraries needed to run the code, eliminating guesswork and ensuring consistency across different operating systems.

B. Local Audit Integration

In clinical applications, data traceability is a critical component of healthcare compliance. While external observability platforms like LangSmith provide valuable trace analysis, the project recognized the need for a persistent, local record of agent executions. The team developed a local audit infrastructure, making refinements to the audit logging files to ensure local trace capturing remains a reliable fallback. This infrastructure guarantees that every node execution is logged locally, providing a redundant safety net for institutional governance and creating a permanent record of the agent's logic for future clinical review [4].

The agent system is made up of five python files that work together to track event logs and store events in the form of a JSON Lines (JSONL) file. JSONL was specifically chosen for its readability and to easily query runs [5]. The python files and their core function are summarized below:

- **Audit Logger:** Starts logging the event and enforces the schema and creates unique IDs to prevent duplication and idempotency.
- **Decorator:** Captures inputs, outputs, runtimes, and any errors in the current node.
- **Schemas:** Defines the structure that is output in the JSONL file to ensure consistent formatting across all event logs.
- **Trace Writer:** Appends all logged traces to the JSONL file.

Returning to the patient example, every step of this retrieval process is captured by these rigorous auditing mechanisms. When the agent successfully retrieves the penicillin allergy via the MCP and generates the final snapshot, the local trace logger automatically writes a permanent record of the execution. This JSONL file documents several event ids, start times (seconds), end times (seconds), overall run latency (milliseconds), the clinician's initial prompt, the specific validation nodes triggered, node parameters, and the final output. It will also include information related to the run status (started, completed, or error) and information on the type of error if it occurs.

There are currently some limitations with the current agent system related to event ids, querying, and error logging. The current system uses the set() function in Python to create unique run ids; with the current functionality there is a chance an event id can be duplicated upon system resets. As the JSONL continues to grow querying events will increase in difficulty, a potential future improvement would be to create a dashboard or database tool to improve ease of log review. Finally, there are functionality issues with the current state of error logs. All error types are not logged and if an error occurs there are cases where

the event has a start but no end time. Additional testing and modification of the error system is needed.

V. SYSTEM EVALUATION AND OUTPUT SYNTHESIS

To validate the clinical AI agent's performance, the system's orchestration logic and data retrieval accuracy were evaluated using observability tools and local audit logs. The analysis focused on the agent's ability to process natural language queries, navigate FHIR, and generate a standardized clinical snapshot.

A. Natural Language Query and Intent Capture

The system was tested using complex, multi-part queries to evaluate its natural language processing capabilities. In a representative test case, the user requested medication and allergy information for a specific patient identifier during recent emergency room encounters. The agent successfully captured the user's intent by identifying the target patient, filtering the data for recent encounters, and isolating the specific clinical categories requested. This demonstrates the orchestration layer's ability to maintain clinical context while translating conversational inputs into actionable database queries.

B. Output Synthesis and Formatting

Following the evaluation of multiple output formats, the project finalized a list-based structure for the ER Clinical Snapshot. This format was selected for its high skimmability, allowing frontline nurses and residents to identify immediate risks within seconds. The output clearly delineates patient identity, encounter history, active medications, and documented allergies.

To support hospital administration and audit teams, the system transforms this list-based output directly into structured JSON data. This ensures that the clinical findings mapped in the snapshot are consistently backed by raw, machine-readable data. By utilizing JSON rather than tabular CSV formats, the system provides an accurate, hierarchical record for billing and resource management while maintaining strict data integrity across presentations.

C. Local Audit Implementation and Governance

A core success criterion for this project was the establishment of a secure and compliant framework. To meet this requirement, the team successfully developed and implemented a local audit infrastructure that acts as a redundant safety net for healthcare compliance. This system ensures that every node execution from the validation layer to the snapshot generator is logged locally. By creating a permanent JSONL record of the agent's logic, the infrastructure provides an immutable trail for future clinical review. This guarantees that data governance standards are maintained independently of external trace retention limits, fulfilling the critical need for long term accountability in automated clinical decision support.

VI. CONCLUSION AND FUTURE WORK

A. Conclusion

This project successfully established a foundational, governed AI framework for clinical decision support in the emergency department. By implementing a secure scaffolding architecture utilizing a local Fast Healthcare Interoperability Resources server and the Model Context Protocol, the clinical agent effectively retrieved and synthesized structured patient data. The system generated the ER Clinical Snapshot in approximately 12 seconds, providing a highly skimmable, list-based output that reduced cognitive load by rapidly surfacing critical information such as allergies and active medications. The ER Clinical Snapshot provides a highly skimmable, list-based output that reduces cognitive load by rapidly surfacing critical information such as allergies and active medications. Furthermore, the integration of a local auditing infrastructure ensured that all agent executions were permanently recorded in a structured JSON format. This development, alongside rigorous environment standardization, guarantees that the system is reproducible, compliant, and ready for future expansion.

B. Future Work and Recommendations

While the current framework validates the architectural logic using synthetic data, several key milestones remain for future capstone teams to achieve production readiness.

1) *Multi-Tiered Environment Expansion (Workspaces B and C)*: Workspace A was developed as a environment using synthetic FHIR data to establish the core agent logic. Future implementation of Workspaces B and C will move the project toward production readiness. Workspace B will involve the integration of de-identified real world data to test agent performance against "noisy" clinical records. Workspace C will focus on the transition to a fully HIPAA compliant production environment, enabling the agent to handle Protected Health Information (PHI) with the necessary security controls.

2) *Transition to Live Clinical Data Streams*: The system currently pulls from a HAPI FHIR server populated with synthetic records. Moving from synthetic datasets to live emergency department data feeds will allow future teams to refine the agent's "clinical intuition." Real world data often contains missing fields, non-standard coding, and irregular timing. Tightening the validation nodes to handle these discrepancies is essential for maintaining clinical safety in live settings.

3) *Human-in-the-Loop (HITL) and Interactivity*: The current LangGraph workflow is a directed sequence ending in a generated snapshot. Implementing "Interrupt" nodes within the graph would allow for clinician feedback. For example, the agent could pause to ask a physician for clarification when it finds conflicting allergy data in the records. This ensures the AI remains a supportive tool rather than an autonomous decision maker.

4) *Multimodal Integration (Imaging and Notes)*: The agent currently processes structured FHIR resources such as Medications, Observations, and Allergies. As the project title suggests, the end goal is a multimodal agent. Future iterations should incorporate unstructured data by using natural language processing to parse physician notes. Additionally, integrating metadata or radiology report summaries from medical imaging into the clinical snapshot would provide a more holistic view of the patient.

5) *EHR Integration (SMART-on-FHIR)*: The project currently runs as a standalone implementation on a sponsor managed server. To place the AI agent directly into the clinician's existing workflow, future teams should develop frontend components that allow the agent to be launched directly within an Electronic Health Record (EHR) system. Utilizing the SMART-on-FHIR standard would facilitate this seamless integration.

VII. STUDENT CONTRIBUTIONS

The following table outlines the distribution of responsibilities and the collaborative effort required to complete this project. To ensure technical accuracy and institutional governance, the team utilized a lead and support model for each major milestone. A Lead Contributor indicates the primary architect or manager of a specific task, while Supporting Contributor identifies team members who provided peer review, testing assistance, or documentation support. This structure highlights that although specific individuals drove key technical workstreams, the final system represents a unified effort across all project domains.

TABLE II. CONTRIBUTION SUMMARY

Activity	Lead Contributor(s)	Supporting Contributor(s)
Exploratory Data Analysis	All Team Members	All Team Members

Agent & FHIR/MCP Logic	Heejeong (Angie) Yoon	Hannah Richardson, Chelsey Blowe, Matthew Lang
Audit Infrastructure	Hannah Richardson	Heejeong (Angie) Yoon, Chelsey Blowe, Matthew Lang
GitHub Architecture	All Team Members	All Team Members
Project Documentation	All Team Members	All Team Members
Project Management & Communication	Matthew Lang	Chelsey Blowe, Hannah Richardson, Heejeong (Angie) Yoon
Academic Synthesis & Deliverables	Matthew Lang	Chelsey Blowe, Hannah Richardson, Heejeong (Angie) Yoon

VIII. REFERENCES

[1] Health Level Seven International, "Home - US Core implementation guide v8.0.1." [Online]. Available: <https://hl7.org/fhir/us/core/>. Accessed: April 28, 2026.

[2] Smile CDR Inc., "HAPI FHIR." [Online]. Available: <https://hapi.fhir.org/>. Accessed: April 28, 2026.

[3] LangChain, "Agents - docs by LangChain." [Online]. Available: <https://docs.langchain.com/oss/python/langchain/agents>. Accessed: April 28, 2026.

[4] Datadog, "What is audit logging?," Datadog. [Online]. Available: <https://www.datadoghq.com/knowledge-center/audit-logging/>. Accessed: April 28, 2026.

[5] SolarWinds Loggly, " JSON Logging Best Practices." [Online]. Available:<https://www.loggly.com/use-cases/json-logging-best-practices/>. Accessed: April 29, 2026